

AI ASSIST CODING ASSIGNMENT-15.5

Program Name: B. Tech

Assignment Type: Lab

Academic Year: 2025-2026

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Student Name: Sanjay Karupothula

Roll No: 2303A52337

batch: 34

Assignment Number: 15.5 / 24

Lab 15 – Backend API Development: Creating RESTful Services with AI

Lab Objectives

- Understand the fundamentals of RESTful API design.
 - Use AI-assisted coding tools to generate backend services.
 - Implement CRUD (Create, Read, Update, Delete) operations.
 - Document endpoints with comments and auto-generated documentation.
-

Task 1 – Student Attendance API

Prompt Used

Generate a RESTful API for student attendance tracking with CRUD operations using JSON based request and response.

Required Endpoints

GET /attendance → View attendance records

POST /attendance → Mark attendance

PUT /attendance/{id} → Update attendance record
DELETE /attendance/{id} → Remove attendance entry

Code Implementation

```
from flask import Flask, request, jsonify
app = Flask(__name__)

attendance = []

@app.route('/attendance', methods=['GET'])
def get_attendance():
    return jsonify(attendance)

@app.route('/attendance', methods=['POST'])
def mark_attendance():
    data = request.json
    attendance.append(data)
    return jsonify({"message": "Attendance added"})

@app.route('/attendance/<int:id>', methods=['PUT'])
def update_attendance(id):
    if id < len(attendance):
        attendance[id] = request.json
        return jsonify({"message": "Attendance updated"})
    return jsonify({"error": "Record not found"})

@app.route('/attendance/<int:id>', methods=['DELETE'])
def delete_attendance(id):
    if id < len(attendance):
        attendance.pop(id)
        return jsonify({"message": "Attendance deleted"})
    return jsonify({"error": "Record not found"})

if __name__ == '__main__':
    app.run(debug=True)
```

Output

API successfully performs attendance operations.

Screenshot of Execution – Task 1



OneCompiler

main.py + 44g6yu7m

```
1 attendance = []
2
3 def get_attendance():
4     return attendance
5
6 def mark_attendance(record):
7     attendance.append(record)
8     return "Attendance Added"
9
10 def update_attendance(id, record):
11     if id < len(attendance):
12         attendance[id] = record
13         return "Attendance Updated"
14     return "Record Not Found"
15
16 def delete_attendance(id):
17     if id < len(attendance):
18         attendance.pop(id)
19         return "Attendance Deleted"
20     return "Record Not Found"
21
22 print(mark_attendance({"name": "Rahul", "status": "Present"}))
23 print(get_attendance())
24 print(update_attendance(0, {"name": "Rahul", "status": "Absent"}))
25 print(delete_attendance(0))
26
```

Output:
Attendance Added
[{'name': 'Rahul', 'status': 'Present'}]
Attendance Updated
Attendance Deleted

Task 2 – Inventory Management API

Prompt Used

Generate a REST API for warehouse inventory management including validation for stock values.

Required Endpoints

GET /inventory → List all items
POST /inventory → Add new item
PUT /inventory/{id} → Update stock quantity
DELETE /inventory/{id} → Remove an item

Code Implementation

```
from flask import Flask, request, jsonify
app = Flask(__name__)

inventory = []

@app.route('/inventory', methods=['GET'])
def list_items():
    return jsonify(inventory)

@app.route('/inventory', methods=['POST'])
def add_item():
```

```

    item = request.json
    if item.get("stock",0) < 0:
        return jsonify({"error":"Invalid stock value"})
    inventory.append(item)
    return jsonify({"message":"Item added"})

@app.route('/inventory/<int:id>', methods=['PUT'])
def update_stock(id):
    if id < len(inventory):
        inventory[id]['stock'] = request.json.get('stock',
inventory[id]['stock'])
        return jsonify({"message":"Stock updated"})
    return jsonify({"error":"Item not found"})

@app.route('/inventory/<int:id>', methods=['DELETE'])
def delete_item(id):
    if id < len(inventory):
        inventory.pop(id)
        return jsonify({"message":"Item removed"})
    return jsonify({"error":"Item not found"})

if __name__ == '__main__':
    app.run(debug=True)

```

Output

API manages inventory stock levels successfully.

Screenshot of Execution – Task 2



44g6yu7m

```

main.py
1 inventory = []
2
3 def list_items():
4     return inventory
5
6 def add_item(item):
7     if item["stock"] < 0:
8         return "Invalid Stock"
9     inventory.append(item)
10    return "Item Added"
11
12 def update_stock(id, stock):
13     if id < len(inventory):
14         inventory[id]["stock"] = stock
15         return "Stock Updated"
16     return "Item Not Found"
17
18 def delete_item(id):
19     if id < len(inventory):
20         inventory.pop(id)
21         return "Item Deleted"
22     return "Item Not Found"
23
24 print(add_item({"item": "Laptop", "stock": 10}))
25 print(list_items())
26 print(update_stock(0, 5))
27 print(delete_item(0))
28
29

```

Output:

```

Item Added
[{'item': 'Laptop', 'stock': 10}]
Stock Updated
Item Deleted

```

Task 3 – Hotel Room Booking API

Prompt Used

Generate a RESTful API for hotel room booking with error handling for unavailable rooms.

Required Endpoints

GET /rooms → View available rooms
POST /rooms/book → Book a room
GET /rooms/{id} → View booking details
DELETE /rooms/{id} → Cancel booking

Code Implementation

```
from flask import Flask, request, jsonify
app = Flask(__name__)

rooms = {1:"available",2:"available",3:"available"}

@app.route('/rooms', methods=['GET'])
def view_rooms():
    return jsonify(rooms)

@app.route('/rooms/book', methods=['POST'])
def book_room():
    room_id = request.json.get('room')
    if rooms.get(room_id) == "available":
        rooms[room_id] = "booked"
        return jsonify({"message":"Room booked"})
    return jsonify({"error":"Room unavailable"})

@app.route('/rooms/<int:id>', methods=['GET'])
def booking_details(id):
    return jsonify({"room": id, "status": rooms.get(id)})

@app.route('/rooms/<int:id>', methods=['DELETE'])
def cancel_booking(id):
    if rooms.get(id) == "booked":
        rooms[id] = "available"
        return jsonify({"message":"Booking cancelled"})
    return jsonify({"error":"No booking"})
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

Output

API simulates hotel room booking successfully.

Screenshot of Execution – Task 3



OneCompiler

main.py + 44g6yu7rn

```
1 rooms = {'1':'available',2:'available',3:'available'}  
2  
3 def view_rooms():  
4     return rooms  
5  
6 def book_room(id):  
7     if rooms[id] == "available":  
8         rooms[id] = "booked"  
9         return "Room Booked"  
10    return "Room Unavailable"  
11  
12 def cancel_booking(id):  
13     if rooms[id] == "booked":  
14         rooms[id] = "available"  
15         return "Booking Cancelled"  
16    return "No Booking"  
17  
18 print(view_rooms())  
19 print(book_room(1))  
20 print(view_rooms())  
21 print(cancel_booking(1))  
22  
23
```

Output:

{1: 'available', 2: 'available', 3: 'available'}
Room Booked
{1: 'booked', 2: 'available', 3: 'available'}
Booking Cancelled

Task 4 – Online Voting API

Prompt Used

Generate a RESTful API for an online voting system ensuring one vote per user.

Required Endpoints

GET /candidates → List candidates

POST /vote → Cast a vote

GET /results → View voting results

Code Implementation

```
from flask import Flask, request, jsonify  
app = Flask(__name__)
```

```
candidates = {"A":0,"B":0,"C":0}  
voted_users = set()
```

```

@app.route('/candidates', methods=['GET'])
def list_candidates():
    return jsonify(candidates)

@app.route('/vote', methods=['POST'])
def vote():
    user = request.json.get('user')
    candidate = request.json.get('candidate')

    if user in voted_users:
        return jsonify({"error": "User already voted"})

    if candidate in candidates:
        candidates[candidate] += 1
        voted_users.add(user)
        return jsonify({"message": "Vote counted"})

    return jsonify({"error": "Invalid candidate"})

@app.route('/results', methods=['GET'])
def results():
    return jsonify(candidates)

if __name__ == '__main__':
    app.run(debug=True)

```

Output

API simulates voting and displays results correctly.

Screenshot of Execution – Task 4



main.py

+

44g6yu7rn

```

1 candidates = {"A":0,"B":0,"C":0}
2 voted = set()
3
4 def vote(user, candidate):
5     if user in voted:
6         return "Already Voted"
7     candidates[candidate] += 1
8     voted.add(user)
9     return "Vote Counted"
10
11 def results():
12     return candidates
13
14 print(vote("user1", "A"))
15 print(vote("user1", "B"))
16 print(results())
17
18
19

```

Output:

Vote Counted
Already Voted
{'A': 1, 'B': 0, 'C': 0}

Task 5 – Teacher Management API

Prompt Used

Generate a RESTful API for managing teacher records with proper error handling.

Required Endpoints

GET /teachers → List all teachers

POST /teachers → Add a new teacher

PUT /teachers/{id} → Update teacher details

DELETE /teachers/{id} → Remove a teacher

Code Implementation

```
from flask import Flask, request, jsonify
app = Flask(__name__)

teachers = []

@app.route('/teachers', methods=['GET'])
def get_teachers():
    return jsonify(teachers)

@app.route('/teachers', methods=['POST'])
def add_teacher():
    teachers.append(request.json)
    return jsonify({"message": "Teacher added"})

@app.route('/teachers/<int:id>', methods=['PUT'])
def update_teacher(id):
    if id < len(teachers):
        teachers[id] = request.json
        return jsonify({"message": "Teacher updated"})
    return jsonify({"error": "Teacher not found"})

@app.route('/teachers/<int:id>', methods=['DELETE'])
def delete_teacher(id):
    if id < len(teachers):
        teachers.pop(id)
        return jsonify({"message": "Teacher deleted"})
    return jsonify({"error": "Teacher not found"})
```



```
if __name__ == '__main__':  
    app.run(debug=True)
```

Output

API performs CRUD operations for teacher records successfully.

Screenshot of Execution – Task 5



 OneCompiler

main.py

+

44g6yu7m 

```
1 teachers = []  
2  
3 def list_teachers():  
4     return teachers  
5  
6 def add_teacher(t):  
7     teachers.append(t)  
8     return "Teacher Added"  
9  
10 def update_teacher(id,t):  
11     if id < len(teachers):  
12         teachers[id] = t  
13         return "Teacher Updated"  
14     return "Teacher Not Found"  
15  
16 def delete_teacher(id):  
17     if id < len(teachers):  
18         teachers.pop(id)  
19         return "Teacher Deleted"  
20     return "Teacher Not Found"  
21  
22 print(add_teacher({"name":"Mr Rao","subject":"Math"}))  
23 print(list_teachers())  
24 print(update_teacher(0,{"name":"Mr Rao","subject":"Physics"}))  
25 print(delete_teacher(0))  
26
```

Output:

Teacher Added
[{'name': 'Mr Rao', 'subject': 'Math'}]
Teacher Updated
Teacher Deleted

Conclusion

This lab demonstrated how AI-assisted coding tools can be used to design RESTful APIs and implement CRUD operations for different real-world scenarios such as attendance tracking, inventory management, hotel booking, online voting, and teacher management systems.